

Projet Python — Séance 3

pandas, jointures et découverte de numpy

Enseignante : Rahma GARGOURI — 2026-2027

Séance de 3 heures — Projet en groupe

Objectif. Reprendre les taux de la séance 1 et les traiter avec **pandas** (bien plus court et rapide que des boucles) : charger, nettoyer, analyser. Apprendre les **jointures**, découvrir **numpy**, et jeter un œil à **polars**. C'est votre première utilisation de **librairies tierces** (installées avec **pip**).

Comme précédemment, le travail se fait sur une branche **seance-3**, avec une *pull request* en fin de séance. Après chaque petite partie, une boîte bleue rassemble des questions à **rédigier** dans `docs/notes_seance3.md` et à **commiter**. L'IA est autorisée, mais chaque réponse doit être comprise (elle pourra être demandée à l'oral).

Partie A — Découvrir pandas

A.1 — Installer et charger. Installer **pandas** (`pip install pandas`). Charger le CSV des taux (séance 1) dans un **DataFrame** avec `read_csv`, puis l'inspecter : `head()`, `describe()`, et le type de chaque colonne (`dtypes`).

A.2 — Sélectionner et filtrer. Sélectionner une colonne (une **Series**), filtrer des lignes (par exemple les jours où le taux dépasse une valeur donnée). Recalculer la moyenne, le minimum et le maximum avec **pandas** et comparer avec vos fonctions de la séance 2.

Questions — à rédiger et à commiter

1. Qu'est-ce qu'un **DataFrame** ? Une **Series** ?
2. Pourquoi utiliser **pandas** plutôt que des boucles et le module `csv` ? Que signifie « vectorisation » ?
3. Pourquoi déconseille-t-on `iterrows()` ?
4. Comment **pandas** représente-t-il les valeurs manquantes (**NaN**) ? Conséquences sur les moyennes ?

Partie B — Nettoyer, consolider et analyser

B.1 — Comblé les trous. Les taux ne sont publiés que les jours ouvrés. Réindexer le **DataFrame** sur un calendrier de jours ouvrés **continu** et propager la dernière valeur connue (*forward-fill*) pour les jours manquants. Justifier ce choix dans les notes.

B.2 — Analyser. Calculer la variation en % sur la période, une **moyenne mobile**, et rééchantillonner en données **mensuelles** (`resample`).

Questions — à rédiger et à commiter

1. C'est quoi le *forward-fill* ? Quelle hypothèse fait-on en l'utilisant ?
2. À quoi sert `resample` ? Que signifie une moyenne mobile ?

Partie C — Les jointures

Une **jointure** réunit deux tables côte à côte en associant leurs lignes sur une **colonne commune** appelée **clé**. Avec **pandas**, cela s'écrit `pd.merge(gauche, droite, on="cle", how=...)`, où le paramètre **how** choisit le **type** de jointure — c'est-à-dire quoi faire des lignes qui n'ont **pas** de correspondance dans l'autre table.

Les quatre types :

- **inner** : ne garde que les clés présentes dans **les deux** tables (l'intersection) ;
- **left** : garde **toutes** les lignes de la table de **gauche** ; les colonnes de droite valent **NaN** là où il n'y a pas de correspondance ;
- **right** : garde toutes les lignes de la table de **droite** ; les colonnes de gauche valent **NaN** là où il n'y a pas de correspondance ;
- **outer** : garde **toutes** les lignes des deux tables (l'union) ; **NaN** de chaque côté là où il manque une correspondance.

Exemple. Table de gauche avec les clés {1, 2} et table de droite avec les clés {2, 3} :

how	Clés conservées dans le résultat
inner	{2}
left	{1, 2} — la clé 1 a des NaN du côté droit
right	{2, 3} — la clé 3 a des NaN du côté gauche
outer	{1, 2, 3} — NaN là où il manque une correspondance

Attention à la duplication. Si une clé apparaît **plusieurs fois** dans une table, la jointure produit **toutes les combinaisons** possibles (par exemple 2 lignes à gauche et 3 à droite pour la même clé donnent 6 lignes). Vérifiez toujours le nombre de lignes après un **merge**.

C.1 — Réunir deux séries. Récupérer les taux des **deux devises de votre groupe** (deux **DataFrame**, un par devise) et les réunir en un seul **DataFrame** indexé par date à l'aide de **merge** (jointure sur la date). Sur la même période, les dates coïncident : la jointure est complète.

C.2 — Les types de jointure. Reproduire l'exemple ci-dessus avec deux petites tables aux clés **partiellement communes**, et comparer les jointures **inner**, **left**, **right** et **outer** : observer ce que chacune conserve et où apparaissent des **NaN**.

C.3 — merge, join ou concat ? Distinguer trois façons d'assembler des tables : **merge** joint sur une **colonne clé**, **join** joint sur l'**index**, et **concat** **empile** les tables (les unes sous les autres, ou côte à côte) sans apparier de clé.

Questions — à rédiger et à commiter

1. Rappelez les quatre types de jointure (**inner**, **left**, **right**, **outer**) : que garde chacune ?
2. Sur quelle colonne (**clé**) joint-on ? Que devient une ligne sans correspondance dans l'autre table ?
3. Pour réunir deux séries de taux sur la même période, quelle jointure choisir et pourquoi ?
4. Quelle différence entre **merge** et **concat** ? Dans quel cas utiliser l'un ou l'autre ?

Partie D — Un peu de numpy (et aperçu de polars)

D.1 — Du DataFrame au tableau numpy. Convertir une colonne de taux en tableau **numpy** (`to_numpy()`). Faire des calculs **vectorisés** sans boucle : moyenne, écart-type, variations jour à jour.

D.2 — Comprendre numpy. Découvrir la notion de `dtype` (le type des éléments du tableau) et de *broadcasting* (opération entre un tableau et un nombre, par exemple `taux * 100`).

D.3 — Aperçu de polars. Refaire deux ou trois calculs (moyenne, variation, ou un regroupement) en `polars`, et comparer la syntaxe et le temps d'exécution avec `pandas`.

Questions — à rédiger et à commiter

1. Différence entre une liste Python et un tableau `numpy` (`ndarray`) ? C'est quoi un `dtype` ? Le *broadcasting* ?
2. Pourquoi `numpy` est-il plus rapide que des boucles Python ?
3. Différences entre `pandas` et `polars` (langage d'implémentation, usage des cœurs, évaluation paresseuse) ? Quand choisir l'un ou l'autre ?

Livrables de la séance (commit + push + PR avant la fin)

`analyse.py` (chargement, nettoyage *forward-fill*, variation, moyenne mobile, `resample`) — la jointure des deux devises et l'exemple des quatre types de jointure — les calculs `numpy` et l'aperçu `polars` — `docs/notes_seance3.md` (comparatif avec la séance 2 + réponses aux questions) — **PR de la séance 3** ouverte.